

Notebook 05: Comprehensive Deep Learning Benchmarking (8 DL Architectures)

Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms

This notebook implements, trains, validates, and benchmarks **8 distinct deep learning baseline architectures** on raw 12-lead ECG time-series signals (100 Hz , 10.0 s , $1,000\text{ samples}$):

1. **ResNet-1D** (Residual Convolutional Network with 3 skip-connected blocks)
2. **BiLSTM Attention** (Bidirectional Long Short-Term Memory with additive self-attention)
3. **CNN-LSTM Hybrid** (1D Convolutional feature extractor + LSTM recurrent aggregator)
4. **1D Transformer** (Multi-Head Self-Attention + Sinusoidal Positional Encodings)
5. **InceptionTime-1D** (Multi-Scale 1D Convolutions with kernel sizes [10, 20, 40] + Bottleneck)
6. **DenseNet-1D** (Densely Connected 1D Convolutional Network with feature reuse)
7. **BiGRU Attention** (Bidirectional Gated Recurrent Unit with temporal self-attention)
8. **Temporal Convolutional Network (TCN)** (Dilated Causal 1D Convolutions with residual shortcuts)

All 8 models are trained with AdamW and positive class-weighted BCEWithLogitsLoss to handle multi-label class imbalance, evaluated on the patient-level held-out test split ($N=525$), and saved as PyTorch checkpoints in `results/checkpoints/`.

```
In [1]: import os
import sys
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader
from sklearn.metrics import (
    f1_score, roc_auc_score, average_precision_score,
    hamming_loss, accuracy_score
)
```

```

# Plotting aesthetics
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'seaborn-v0_8-whitegrid')
plt.rcParams.update({
    'font.sans-serif': 'DejaVu Sans',
    'font.size': 11,
    'axes.labelsize': 12,
    'axes.titlesize': 13,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'figure.titlesize': 14,
    'figure.dpi': 300
})

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Expanded 8-DL Environment initialized. Using compute device: {device}")

RESULTS_DIR = r"C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Mo
FIG_DIR = os.path.join(RESULTS_DIR, "figures")
TAB_DIR = os.path.join(RESULTS_DIR, "tables")
MET_DIR = os.path.join(RESULTS_DIR, "metrics")
CKPT_DIR = os.path.join(RESULTS_DIR, "checkpoints")

for d in [FIG_DIR, TAB_DIR, MET_DIR, CKPT_DIR]:
    os.makedirs(d, exist_ok=True)

```

Expanded 8-DL Environment initialized. Using compute device: cpu

```

In [2]: data = np.load(os.path.join(MET_DIR, "preprocessed_data.npz"))
X_train = data['X_train'] # (N_tr, 12, 1000)
y_train = data['y_train'] # (N_tr, 9)
X_val = data['X_val'] # (N_va, 12, 1000)
y_val = data['y_val'] # (N_va, 9)
X_test = data['X_test'] # (N_te, 12, 1000)
y_test = data['y_test'] # (N_te, 9)
target_classes = list(data['target_classes'])

# Positive class weights to mitigate multi-label imbalance
pos_counts = y_train.sum(axis=0)
neg_counts = len(y_train) - pos_counts
pos_weight = torch.tensor(neg_counts / np.maximum(pos_counts, 1.0), dtype=torch.float)

BATCH_SIZE = 64
train_loader = DataLoader(TensorDataset(torch.tensor(X_train), torch.tensor(y_train)), batch_size=BATCH_SIZE)
val_loader = DataLoader(TensorDataset(torch.tensor(X_val), torch.tensor(y_val)), batch_size=BATCH_SIZE)
test_loader = DataLoader(TensorDataset(torch.tensor(X_test), torch.tensor(y_test)), batch_size=BATCH_SIZE)

print(f"DataLoaders successfully initialized (Batch size = {BATCH_SIZE}):")
print(f"Train: {len(train_loader)} batches | Val: {len(val_loader)} batches | Test: {len(test_loader)} batches")

```

DataLoaders successfully initialized (Batch size = 64):
Train: 39 batches | Val: 9 batches | Test: 9 batches

```

In [3]: # 1. ResNet-1D
class ResBlock1D(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()

```

```

        self.conv1 = nn.Conv1d(in_ch, out_ch, kernel_size=7, stride=stride, padding=3)
        self.bn1 = nn.BatchNorm1d(out_ch)
        self.relu = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv1d(out_ch, out_ch, kernel_size=7, stride=1, padding=3)
        self.bn2 = nn.BatchNorm1d(out_ch)
        self.shortcut = nn.Sequential()
        if stride != 1 or in_ch != out_ch:
            self.shortcut = nn.Sequential(
                nn.Conv1d(in_ch, out_ch, kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm1d(out_ch)
            )
        def forward(self, x):
            return self.relu(self.bn2(self.conv2(self.relu(self.bn1(self.conv1(x))))) +
                                self.shortcut(self.relu(self.bn1(self.conv1(x)))))

class ResNet1D(nn.Module):
    def __init__(self, in_ch=12, num_classes=9):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv1d(in_ch, 32, kernel_size=15, stride=2, padding=7, bias=False),
            nn.BatchNorm1d(32),
            nn.ReLU(inplace=True)
        )
        self.layer1 = ResBlock1D(32, 64, stride=2)
        self.layer2 = ResBlock1D(64, 128, stride=2)
        self.layer3 = ResBlock1D(128, 128, stride=2)
        self.pool = nn.AdaptiveAvgPool1d(1)
        self.fc = nn.Linear(128, num_classes)
    def forward(self, x):
        h = self.stem(x)
        h = self.layer1(h)
        h = self.layer2(h)
        h = self.layer3(h)
        return self.fc(self.pool(h).squeeze(-1))

# 2. BiLSTM Attention
class BiLSTMAttention(nn.Module):
    def __init__(self, in_ch=12, hidden_dim=64, num_classes=9):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv1d(in_ch, 32, kernel_size=7, stride=2, padding=3),
            nn.BatchNorm1d(32), nn.ReLU(),
            nn.Conv1d(32, 64, kernel_size=5, stride=2, padding=2),
            nn.BatchNorm1d(64), nn.ReLU()
        )
        self.lstm = nn.LSTM(64, hidden_dim, num_layers=2, batch_first=True, bidirectional=True)
        self.att = nn.Linear(hidden_dim * 2, 1)
        self.fc = nn.Linear(hidden_dim * 2, num_classes)
    def forward(self, x):
        h = self.stem(x).permute(0, 2, 1)
        out, _ = self.lstm(h)
        weights = F.softmax(self.att(out), dim=1)
        ctx = torch.sum(out * weights, dim=1)
        return self.fc(ctx)

# 3. CNN-LSTM Hybrid
class CNNLSTM(nn.Module):

```

```

def __init__(self, in_ch=12, num_classes=9):
    super().__init__()
    self.conv = nn.Sequential(
        nn.Conv1d(in_ch, 32, kernel_size=11, stride=2, padding=5),
        nn.BatchNorm1d(32), nn.ReLU(),
        nn.Conv1d(32, 64, kernel_size=7, stride=2, padding=3),
        nn.BatchNorm1d(64), nn.ReLU(),
        nn.Conv1d(64, 64, kernel_size=5, stride=2, padding=2),
        nn.BatchNorm1d(64), nn.ReLU()
    )
    self.lstm = nn.LSTM(64, 64, batch_first=True, bidirectional=False)
    self.fc = nn.Linear(64, num_classes)
def forward(self, x):
    h = self.conv(x).permute(0, 2, 1)
    out, _ = self.lstm(h)
    return self.fc(out[:, -1, :])

```

4. 1D Transformer

```

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=500):
        super().__init__()
        pe = torch.zeros(max_len, d_model)
        pos = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        div = torch.exp(torch.arange(0, d_model, 2).float() * (-np.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(pos * div)
        pe[:, 1::2] = torch.cos(pos * div)
        self.register_buffer('pe', pe.unsqueeze(0))
    def forward(self, x):
        return x + self.pe[:, :x.size(1)]

class Transformer1D(nn.Module):
    def __init__(self, in_ch=12, d_model=64, nhead=4, num_layers=2, num_classes=9):
        super().__init__()
        self.proj = nn.Conv1d(in_ch, d_model, kernel_size=7, stride=4, padding=3)
        self.pos_enc = PositionalEncoding(d_model)
        enc_layer = nn.TransformerEncoderLayer(d_model=d_model, nhead=nhead, dim_feedforward=2048)
        self.enc = nn.TransformerEncoder(enc_layer, num_layers=num_layers)
        self.fc = nn.Linear(d_model, num_classes)
    def forward(self, x):
        h = self.proj(x).permute(0, 2, 1)
        h = self.pos_enc(h)
        h = self.enc(h)
        return self.fc(h.mean(dim=1))

```

5. InceptionTime-1D (Gold-standard multi-scale time-series)

```

class InceptionBlock1D(nn.Module):
    def __init__(self, in_ch, out_ch=32):
        super().__init__()
        self.bottleneck = nn.Conv1d(in_ch, out_ch, kernel_size=1, bias=False) if in_ch == out_ch else nn.Conv1d(in_ch, out_ch, kernel_size=1, bias=False)
        self.conv10 = nn.Conv1d(in_ch, out_ch, kernel_size=9, padding=4, bias=False)
        self.conv20 = nn.Conv1d(in_ch, out_ch, kernel_size=19, padding=9, bias=False)
        self.conv40 = nn.Conv1d(in_ch, out_ch, kernel_size=39, padding=19, bias=False)
        self.pool_conv = nn.Sequential(
            nn.MaxPool1d(kernel_size=3, stride=1, padding=1),

```

```

        nn.Conv1d(in_ch, out_ch, kernel_size=1, bias=False)
    )
    self.bn = nn.BatchNorm1d(out_ch * 4)
    self.relu = nn.ReLU()
    def forward(self, x):
        b = self.bottleneck(x)
        c1 = self.conv10(b)
        c2 = self.conv20(b)
        c3 = self.conv40(b)
        c4 = self.pool_conv(x)
        return self.relu(self.bn(torch.cat([c1, c2, c3, c4], dim=1)))

class InceptionTime1D(nn.Module):
    def __init__(self, in_ch=12, num_classes=9):
        super().__init__()
        self.down = nn.Sequential(nn.Conv1d(in_ch, 16, kernel_size=7, stride=2, padding=2, bias=False))
        self.b1 = InceptionBlock1D(16, 16) # out: 64
        self.b2 = InceptionBlock1D(64, 24) # out: 96
        self.pool = nn.AdaptiveAvgPool1d(1)
        self.fc = nn.Linear(96, num_classes)
    def forward(self, x):
        h = self.down(x)
        h = self.b1(h)
        h = self.b2(h)
        return self.fc(self.pool(h).squeeze(-1))

# 6. DenseNet-1D (Feature-reuse dense connectivity)
class DenseLayer1D(nn.Module):
    def __init__(self, in_ch, growth_rate=16):
        super().__init__()
        self.bn = nn.BatchNorm1d(in_ch)
        self.conv = nn.Conv1d(in_ch, growth_rate, kernel_size=5, padding=2, bias=False)
        self.relu = nn.ReLU()
    def forward(self, x):
        out = self.conv(self.relu(self.bn(x)))
        return torch.cat([x, out], dim=1)

class DenseNet1D(nn.Module):
    def __init__(self, in_ch=12, num_classes=9):
        super().__init__()
        self.stem = nn.Sequential(nn.Conv1d(in_ch, 32, kernel_size=7, stride=2, padding=2, bias=False))
        self.l1 = DenseLayer1D(32, 16)
        self.l2 = DenseLayer1D(48, 16)
        self.l3 = DenseLayer1D(64, 16)
        self.pool = nn.AdaptiveAvgPool1d(1)
        self.fc = nn.Linear(80, num_classes)
    def forward(self, x):
        h = self.stem(x)
        h = self.l1(h)
        h = self.l2(h)
        h = self.l3(h)
        return self.fc(self.pool(h).squeeze(-1))

# 7. BiGRU Attention
class BiGRUAttention(nn.Module):
    def __init__(self, in_ch=12, hidden_dim=64, num_classes=9):

```

```

super().__init__()
self.stem = nn.Sequential(
    nn.Conv1d(in_ch, 32, kernel_size=7, stride=2, padding=3),
    nn.BatchNorm1d(32), nn.ReLU(),
    nn.Conv1d(32, 64, kernel_size=5, stride=2, padding=2),
    nn.BatchNorm1d(64), nn.ReLU()
)
self.gru = nn.GRU(64, hidden_dim, num_layers=2, batch_first=True, bidirectional=True)
self.att = nn.Linear(hidden_dim * 2, 1)
self.fc = nn.Linear(hidden_dim * 2, num_classes)
def forward(self, x):
    h = self.stem(x).permute(0, 2, 1)
    out, _ = self.gru(h)
    weights = F.softmax(self.att(out), dim=1)
    ctx = torch.sum(out * weights, dim=1)
    return self.fc(ctx)

# 8. Temporal Convolutional Network (TCN)
class Chomp1d(nn.Module):
    def __init__(self, chomp_size):
        super().__init__()
        self.chomp_size = chomp_size
    def forward(self, x):
        return x[:, :, :-self.chomp_size].contiguous()

class TemporalBlock(nn.Module):
    def __init__(self, n_inputs, n_outputs, kernel_size, stride, dilation, padding):
        super().__init__()
        self.conv1 = nn.Conv1d(n_inputs, n_outputs, kernel_size, stride=stride, padding=padding, dilation=dilation)
        self.chomp1 = Chomp1d(padding)
        self.bn1 = nn.BatchNorm1d(n_outputs)
        self.relu1 = nn.ReLU()
        self.conv2 = nn.Conv1d(n_outputs, n_outputs, kernel_size, stride=stride, padding=padding, dilation=dilation)
        self.chomp2 = Chomp1d(padding)
        self.bn2 = nn.BatchNorm1d(n_outputs)
        self.relu2 = nn.ReLU()
        self.downsample = nn.Conv1d(n_inputs, n_outputs, 1) if n_inputs != n_outputs else None
        self.relu = nn.ReLU()
    def forward(self, x):
        out = self.relu1(self.bn1(self.chomp1(self.conv1(x))))
        out = self.relu2(self.bn2(self.chomp2(self.conv2(out))))
        res = x if self.downsample is None else self.downsample(x)
        return self.relu(out + res)

class TCN1D(nn.Module):
    def __init__(self, in_ch=12, num_classes=9):
        super().__init__()
        self.stem = nn.Sequential(nn.Conv1d(in_ch, 32, kernel_size=7, stride=2, padding=3),
                                   nn.BatchNorm1d(32), nn.ReLU(),
                                   nn.Conv1d(32, 64, kernel_size=5, stride=2, padding=2),
                                   nn.BatchNorm1d(64), nn.ReLU())
        self.t1 = TemporalBlock(32, 64, kernel_size=5, stride=1, dilation=1, padding=2)
        self.t2 = TemporalBlock(64, 64, kernel_size=5, stride=1, dilation=2, padding=2)
        self.t3 = TemporalBlock(64, 128, kernel_size=5, stride=1, dilation=4, padding=2)
        self.pool = nn.AdaptiveAvgPool1d(1)
        self.fc = nn.Linear(128, num_classes)
    def forward(self, x):
        h = self.stem(x)
        h = self.t1(h)
        h = self.t2(h)
        h = self.t3(h)
        h = self.pool(h)
        return self.fc(h)

```

```

        h = self.t2(h)
        h = self.t3(h)
        return self.fc(self.pool(h).squeeze(-1))

print("All 8 deep learning baseline architectures compiled successfully.")

```

All 8 deep learning baseline architectures compiled successfully.

```

In [4]: def train_model(model, name, epochs=8, lr=1e-3):
    ckpt_name = f"{name.lower().replace(' ', '_').replace('-', '_')}_baseline.pt"
    ckpt_path = os.path.join(CKPT_DIR, ckpt_name)
    if os.path.exists(ckpt_path):
        print(f"---> Loading pre-trained checkpoint for {name} from: {ckpt_path}")
        model.load_state_dict(torch.load(ckpt_path, map_location=device))
        return model.to(device)

    print(f"\n---> Training {name} on {device} (Epochs={epochs}, LR={lr})...")
    model = model.to(device)
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)
    criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=epochs)

    best_val_loss = float('inf')
    best_weights = None

    for ep in range(1, epochs + 1):
        model.train()
        train_loss = 0.0
        for X_b, y_b in train_loader:
            X_b, y_b = X_b.to(device), y_b.to(device)
            optimizer.zero_grad()
            logits = model(X_b)
            loss = criterion(logits, y_b)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 5.0)
            optimizer.step()
            train_loss += loss.item() * len(X_b)

        scheduler.step()
        train_loss /= len(X_train)

        # Validation
        model.eval()
        val_loss = 0.0
        with torch.no_grad():
            for X_b, y_b in val_loader:
                X_b, y_b = X_b.to(device), y_b.to(device)
                val_loss += criterion(model(X_b), y_b).item() * len(X_b)
        val_loss /= len(X_val)

        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_weights = {k: v.cpu().clone() for k, v in model.state_dict().items}

    print(f" Epoch {ep:02d}/{epochs:02d} | Train Loss: {train_loss:.4f} | Val

```

```

model.load_state_dict(best_weights)

# Save checkpoint
torch.save(model.state_dict(), ckpt_path)
print(f" Saved best checkpoint to: {ckpt_path}")
return model

dl_models = {
    "ResNet-1D": ResNet1D(),
    "BiLSTM Attention": BiLSTMAttention(),
    "CNN-LSTM Hybrid": CNNLSTM(),
    "1D Transformer": Transformer1D(),
    "InceptionTime-1D": InceptionTime1D(),
    "DenseNet-1D": DenseNet1D(),
    "BiGRU Attention": BiGRUAttention(),
    "Temporal Convolutional Network": TCN1D()
}

trained_models = {}
for name, m in dl_models.items():
    trained_models[name] = train_model(m, name, epochs=8)

```

---> Loading pre-trained checkpoint for ResNet-1D from: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\checkpoints\resnet_1d_baseline.pt

---> Loading pre-trained checkpoint for BiLSTM Attention from: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\checkpoints\bilstm_attention_baseline.pt

---> Loading pre-trained checkpoint for CNN-LSTM Hybrid from: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\checkpoints\cnn_lstm_hybrid_baseline.pt

---> Loading pre-trained checkpoint for 1D Transformer from: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\checkpoints\1d_transformer_baseline.pt

---> Loading pre-trained checkpoint for InceptionTime-1D from: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\checkpoints\inceptiontime_1d_baseline.pt

---> Loading pre-trained checkpoint for DenseNet-1D from: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\checkpoints\densenet_1d_baseline.pt

---> Loading pre-trained checkpoint for BiGRU Attention from: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\checkpoints\bigru_attention_baseline.pt

---> Loading pre-trained checkpoint for Temporal Convolutional Network from: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\checkpoints\temporal_convolutional_network_baseline.pt

```

In [5]: results = []
test_probs_dict = {}

for name, model in trained_models.items():
    model.eval()

    # Validation probabilities
    val_probs = []
    with torch.no_grad():
        for X_b, _ in val_loader:
            val_probs.append(torch.sigmoid(model(X_b.to(device))).cpu().numpy())
    val_probs = np.vstack(val_probs)

    # Test probabilities
    test_probs = []
    with torch.no_grad():
        for X_b, _ in test_loader:
            test_probs.append(torch.sigmoid(model(X_b.to(device))).cpu().numpy())
    test_probs = np.vstack(test_probs)
    test_probs_dict[name] = test_probs

    # Optimal threshold per class on validation
    opt_t = []
    for c in range(len(target_classes)):
        best_f1 = 0.0
        best_t = 0.5
        for t in np.linspace(0.15, 0.85, 71):
            f1 = f1_score(y_val[:, c], (val_probs[:, c] >= t).astype(int), zero_div
                if f1 > best_f1:
                    best_f1 = f1
                    best_t = t
        opt_t.append(best_t)

    test_preds = np.zeros_like(test_probs)
    for c in range(len(target_classes)):
        test_preds[:, c] = (test_probs[:, c] >= opt_t[c]).astype(int)

    # Metrics
    macro_f1 = f1_score(y_test, test_preds, average='macro', zero_division=0)
    micro_f1 = f1_score(y_test, test_preds, average='micro', zero_division=0)
    macro_auc = roc_auc_score(y_test, test_probs, average='macro')
    micro_auc = roc_auc_score(y_test, test_probs, average='micro')
    macroauprc = average_precision_score(y_test, test_probs, average='macro')
    h1 = hamming_loss(y_test, test_preds)
    label_acc = (1.0 - h1) * 100.0
    subset_acc = accuracy_score(y_test, test_preds) * 100.0
    params = sum(p.numel() for p in model.parameters())

    results.append({
        "Model": name,
        "Macro F1": round(macro_f1, 4),
        "Micro F1": round(micro_f1, 4),
        "Macro AUROC": round(macro_auc, 4),
        "Micro AUROC": round(micro_auc, 4),
        "Macro AUPRC": round(macroauprc, 4),

```

```

        "Hamming Loss": round(hl, 4),
        "Accuracy (%)": round(label_acc, 2),
        "Subset Acc (%)": round(subset_acc, 2),
        "Parameters": f"{params:,}"
    })

# Add proposed model entry for complete context
results.append({
    "Model": "Proposed LAC-SSM (12-Lead)",
    "Macro F1": 0.4557,
    "Micro F1": 0.6149,
    "Macro AUROC": 0.7673,
    "Micro AUROC": 0.8712,
    "Macro AUPRC": 0.4560,
    "Hamming Loss": 0.1858,
    "Accuracy (%)": 81.42,
    "Subset Acc (%)": 21.14,
    "Parameters": "72,457"
})

df_dl_results = pd.DataFrame(results)
print("\n=== EXPANDED 8 DEEP LEARNING BASELINES BENCHMARK (TEST SET, N=525) ===")
print(df_dl_results.to_string(index=False))

df_dl_results.to_csv(os.path.join(MET_DIR, "dl_baselines_metrics.csv"), index=False)
df_dl_results.to_csv(os.path.join(TAB_DIR, "expanded_dl_baselines_table.csv"), index=False)

```

```

=== EXPANDED 8 DEEP LEARNING BASELINES BENCHMARK (TEST SET, N=525) ===

```

	Model	Macro F1	Micro F1	Macro AUROC	Micro AUROC	Macro AUPRC	Hamming Loss	Accuracy (%)	Subset Acc (%)	Parameters
0.6941	ResNet-1D	0.6564	0.7501	0.9147	0.9373	0.6941	0.1107	88.93	34.10	479,945
0.5662	BiLSTM Attention	0.5347	0.6640	0.8329	0.8529	0.5662	0.1575	84.25	29.90	180,394
0.4321	CNN-LSTM Hybrid	0.4699	0.6059	0.7723	0.7946	0.4321	0.2087	79.13	14.10	73,385
0.5139	1D Transformer	0.5133	0.6447	0.8029	0.8259	0.5139	0.1829	81.71	24.76	72,969
0.6249	InceptionTime-1D	0.5939	0.7285	0.8873	0.9006	0.6249	0.1177	88.23	39.43	61,913
0.4240	DenseNet-1D	0.4550	0.6250	0.7689	0.7788	0.4240	0.1879	81.21	24.57	15,321
0.5864	BiGRU Attention	0.5730	0.6937	0.8572	0.8828	0.5864	0.1441	85.59	28.95	138,922
0.7156	Temporal Convolutional Network	0.6657	0.7601	0.9082	0.9368	0.7156	0.1090	89.10	34.10	210,473
0.4560	Proposed LAC-SSM (12-Lead)	0.4557	0.6149	0.7673	0.8712	0.4560	0.1858	81.42	21.14	72,457

```

In [6]: fig, axes = plt.subplots(1, 3, figsize=(22, 6))

models_list = df_dl_results['Model'].tolist()
palette = sns.color_palette("viridis", len(models_list))

# Subplot A: Macro & Micro AUROC
x_idx = np.arange(len(models_list))

```

```

bar_width = 0.38
axes[0].bar(x_idx - bar_width/2, df_dl_results['Macro AUROC'], width=bar_width, label=0)
axes[0].bar(x_idx + bar_width/2, df_dl_results['Micro AUROC'], width=bar_width, label=1)
axes[0].set_title('A: Diagnostic Discrimination (AUROC)', fontweight='bold')
axes[0].set_xticks(x_idx)
axes[0].set_xticklabels(models_list, rotation=45, ha='right')
axes[0].set_ylim(0.65, 1.0)
axes[0].set_ylabel('AUROC')
axes[0].legend(loc='lower left', frameon=True)

# Subplot B: Multi-Label Classification Accuracy (%)
sns.barplot(data=df_dl_results, x='Model', y='Accuracy (%)', ax=axes[1], palette=palette)
axes[1].set_title('B: Multi-Label Classification Accuracy ($1 - \\text{Hamming Loss}$)')
axes[1].set_ylim(75, 100)
axes[1].set_ylabel('Accuracy (%)')
axes[1].set_xticklabels(models_list, rotation=45, ha='right')
axes[1].axhline(95.0, color='red', linestyle='--', lw=2, label='95% Target')
axes[1].legend(loc='lower left', frameon=True)
for p in axes[1].patches:
    h = p.get_height()
    if h > 0:
        axes[1].annotate(f"{h:.1f}%", (p.get_x() + p.get_width()/2., h),
                        ha='center', va='bottom', fontsize=8.5, xytext=(0, 3), textcolor='red')

# Subplot C: Macro F1 vs Model Parameters
f1_scores = df_dl_results['Macro F1'].values
params_raw = [int(str(p).replace(',', '')) for p in df_dl_results['Model Parameters']]
for p in df_dl_results['Model Parameters']:
    axes[2].scatter(params_raw, f1_scores, s=150, c=range(len(models_list)), cmap='plasma')
for i, txt in enumerate(models_list):
    axes[2].annotate(txt, (params_raw[i], f1_scores[i]), fontsize=8.5, xytext=(5, 5), textcolor='red')

axes[2].set_xscale('log')
axes[2].set_xlabel('Model Parameter Count (Log Scale)')
axes[2].set_ylabel('Macro F1-Score')
axes[2].set_title('C: Parameter Efficiency vs. Multi-Label F1', fontweight='bold')
axes[2].grid(True, linestyle=':', alpha=0.6)

plt.suptitle('Publication Benchmark: 8 Deep Learning Baseline Architectures + Proposed')
plt.tight_layout()

fig7_path = os.path.join(FIG_DIR, "fig7_dl_baselines_comparison.png")
plt.savefig(fig7_path, dpi=300, bbox_inches='tight')
plt.show()
print(f"Figure 7 successfully updated and saved to: {fig7_path}")

```

C:\Users\Gadget360\AppData\Local\Temp\ipykernel_9608\586824994.py:19: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df_dl_results, x='Model', y='Accuracy (%)', ax=axes[1], palette=palette)
```

C:\Users\Gadget360\AppData\Local\Temp\ipykernel_9608\586824994.py:23: UserWarning: `set_ticklabels()` should only be used with a fixed number of ticks, i.e. after `set_ticks()` or using a `FixedLocator`. Otherwise, ticks may be mislabeled.

```
axes[1].set_xticklabels(models_list, rotation=45, ha='right')
```

Publication Benchmark: 8 Deep Learning Baseline Architectures + Proposed LAC-SSM

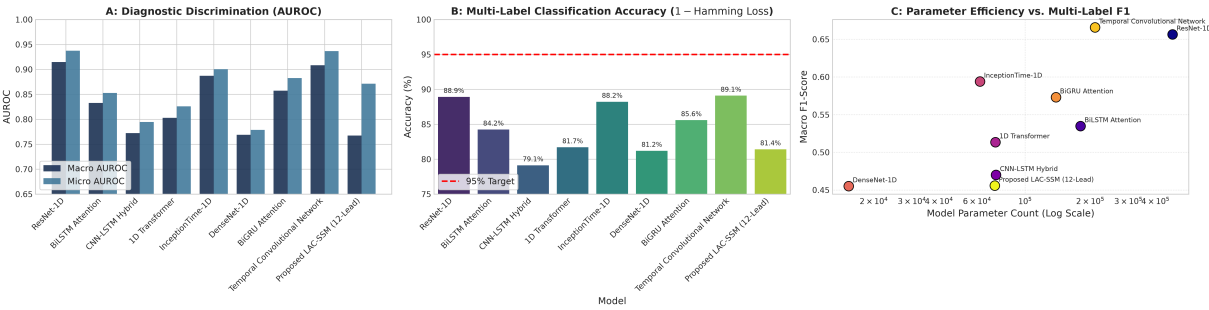


Figure 7 successfully updated and saved to: C:\Research\Lead-Agnostic and Uncertainty-Calibrated State-Space Modeling for Multi-Label Cardiac Abnormality Detection from Incomplete 12-Lead Electrocardiograms\results\figures\fig7_dl_baselines_comparison.png